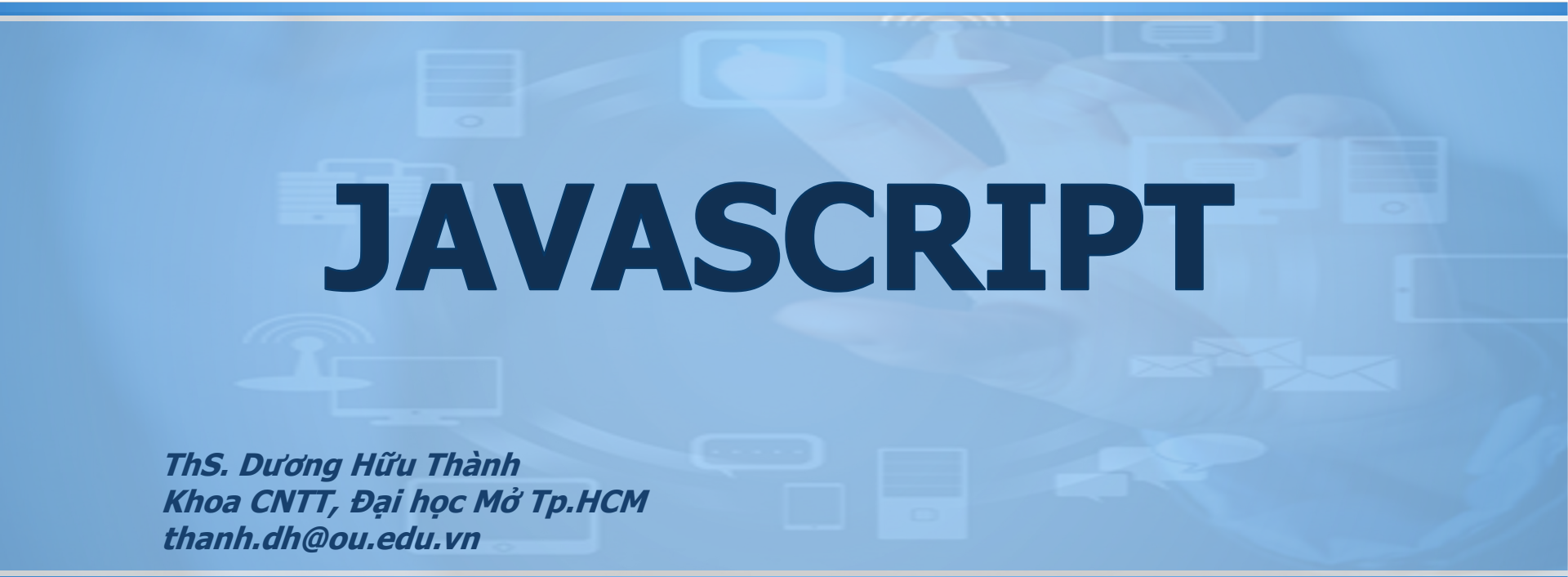


THIẾT KẾ WEB



JAVASCRIPT



ThS. Dương Hữu Thành
Khoa CNTT, Đại học Mở Tp.HCM
thanh.dh@ou.edu.vn

HTML



JS



CSS





Nội dung

- 1. Giới thiệu ngôn ngữ Javascript.**
- 2. Cấu trúc điều khiển.**
- 3. Xử lý sự kiện.**
- 4. Các đối tượng: Number, String, Date, Array.**
- 5. HTML DOM.**
- 6. HTML BOM.**



Giới thiệu ngôn ngữ Javascript



- Javascript là ngôn ngữ lập trình **thông dịch phía client** (client-side) có khả năng hướng đối tượng.
- Javascript xuất hiện đầu tiên trên Netscape 2.0 năm 1995 với tên là LiveScript.
- Javascript có thể
 - Thay đổi nội dung trang web.
 - Thay đổi thuộc tính các thành phần HTML.
 - Thay đổi CSS của các thành phần HTML.
 - Kiểm tra (validate) tính hợp lệ dữ liệu vào của người dùng.



Giới thiệu ngôn ngữ Javascript



- Ưu điểm:
 - Hạn chế tương tác với server.
 - Phản hồi ngay tức thì.
 - Tăng tính tương tác client.
 - Giao diện phong phú hơn.
- Khuyết điểm: Javascript thiếu các đặc trưng sau
 - Không được phép đọc và ghi tập tin.
 - Không thể sử dụng javascript cho các ứng dụng mạng.
 - Không có khả năng multithreading và multiprocessor.



Chèn Javascript vào tài liệu HTML

- Mã nguồn javascript có thể viết **bất cứ đâu** trong tài liệu HTML.
- Nếu cần **xử lý các sự kiện** thì mã nguồn javascript có thể viết trong thành phần **<head>** của tài liệu HTML.

```
<head>
...
<script type="text/javascript">
    document.write("Hello, World!");
</script>
...
</head>
```




Chèn Javascript vào tài liệu HTML

- Nếu cần **tạo ra nội dung** nào đó khi trang nạp lên thì javascript có thể viết trong thành phần **<body>** của tài liệu HTML.

```
<body>
...
<script type="text/javascript">
    document.write("Hello, World!");
</script>
...
</body>
```



Chèn Javascript vào tài liệu HTML

- Gắn tập tin ***.js** chứa mã nguồn javascript vào tài liệu HTML thông qua thẻ **<script>** như sau:

```
<script src="<tập-tin-js>.js" type="text/javascript">  
</script>
```

- Đây là cách thường dùng vì những ưu điểm sau:
 - Tách biệt được HTML và mã nguồn javascript nên chương trình **dễ đọc, dễ bảo trì hơn**.
 - Các tập tin javascript được cache nên **sẽ tăng được tốc độ nạp** trang.



Cú pháp Javascript

- Mỗi câu lệnh **kết thúc bằng dấu chấm phẩy (;)**. Tuy nhiên nếu các lệnh nằm trên các dòng riêng thì có thể bỏ qua chấm phẩy cuối lệnh.
- Phân biệt chữ hoa, chữ thường (**case-sensitive**).
- Để ghi chú trong javascript sử dụng một trong các ký hiệu sau:
 - Ghi chú trên một dòng: **// ...**
 - Ghi chú trên nhiều dòng: **/* ... */**
- Các ghi chú sẽ được bỏ qua bởi trình thông dịch.



Kiểu dữ liệu



- Javascript cung cấp 3 kiểu dữ liệu cơ bản
 - **Number**: kiểu số
 - **String**: kiểu chuỗi
 - **Boolean**: kiểu luận lý (true/false)
- Tất cả số trong javascript đều là số chấm động (floating-point), sử dụng định dạng chấm động 64 bit theo chuẩn IEEE 754.



Biến

- Javascript sử dụng từ khóa **var** để khai báo biến.
- Sử dụng ký hiệu **"="** để gán giá trị cho biến.
- Khi khai báo biến **không cần khai báo kiểu** của biến.
- Biến trong javascript có thể **giữ giá trị của bất cứ kiểu dữ liệu nào** (untyped language).
- Ví dụ:

```
var x = 1, y = true;  
x = "hello";  
Y = 0;
```




Biến

- Tên biến **không trùng** với từ khóa như var, break, boolean, function, ...
- Tên biến **không bắt đầu** bằng các ký số (0..9).
- Tên biến **bắt buộc** bắt đầu bằng ký tự (a..z, A..Z) hoặc dấu gạch chân (underscore), hoặc dấu đôla (\$).
 - Ví dụ: var amount, _totalAmount, \$money;
- Tên biến **phân biệt hoa, thường**.
 - Ví dụ: total và Total là hai biến khác nhau.



Phạm vi biến



- Biến cục bộ (**local**)
 - Biến được **khai báo trong hàm** và chỉ được sử dụng cục bộ trong hàm đó.
 - Biến được tạo khi hàm bắt đầu và bị xóa khi hàm kết thúc.
- Biến toàn cục (**global**):
 - Biến **khai báo ngoài hàm**, các hàm và script trên trang web đều có thể truy cập đến nó.
 - Biến được tạo khi nó được khai báo và bị xóa khi đóng trình duyệt hoặc tab.



Phạm vi biến



- Chú ý:
 - Các **tham số của hàm** được xem là các biến cục bộ trong hàm.
 - Nếu khởi gán biến trong một hàm mà không khai báo với **var** thì nó **tự động trở thành biến toàn cục**. (không áp dụng trong "strict mode")
- Ví dụ:

```
test();  
alert(num);  
  
function test() {  
    num = 5;  
}
```




Các phép toán



- Phép toán số học
 - Cộng (+), trừ (-), nhân (*), chia(/)
 - Lấy phần dư (%)
 - Tăng 1 (++), giảm 1 (--)
- Phép gán: =, +=, -=, *=, /=, %=
- Phép toán luận lý
 - AND (&&)
 - OR (||)
 - NOT (!)



Các phép toán



- Phép so sánh
 - Bằng giá trị (`==`)
 - Khác (`!=`)
 - Bằng giá trị và kiểu (`===`)
 - Khác giá trị hoặc kiểu (`!==`)
 - Lớn hơn (`>`), lớn hơn hoặc bằng (`>=`)
 - Nhỏ hơn (`<`), nhỏ hơn hoặc bằng (`<=`)



Toán tử typeof

- Toán tử **typeof** dùng xác định kiểu dữ liệu của một toán hạng.
- Kết quả của typeof là một chuỗi như sau:
 - "number"
 - "string"
 - "boolean"
 - "object"
 - "function"
 - "undefined"



Xuất dữ liệu trong Javascript



- Javascript hiển thị dữ liệu theo một trong các cách sau:
 - **innerHTML()**: ghi vào thành phần HTML.
 - **document.write()**: xuất ra một HTML.
 - **window.alert()**: xuất ra alert box.
 - **console.log()**: xuất ra console của trình duyệt.
- Ví dụ:

```
alert("Hello, World!");  
console.log("Hello, World!");  
document.write("Hello, World!");
```


JAVASCRIPT

CẤU TRÚC ĐIỀU KHIỂN

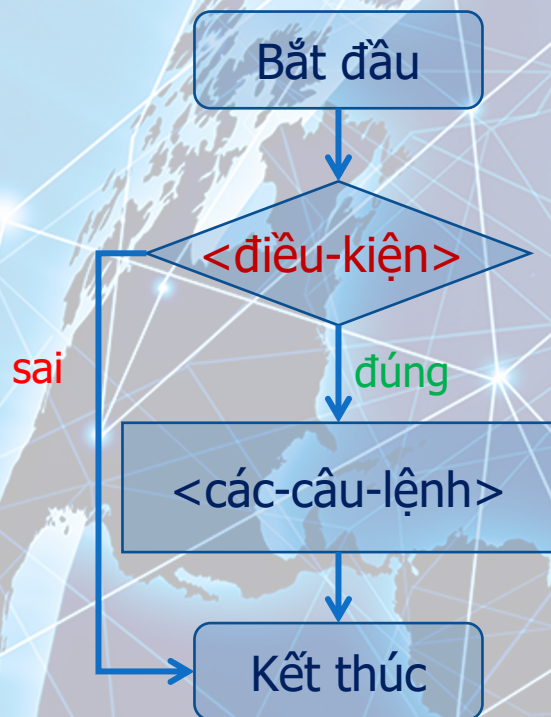


Lệnh rẽ nhánh



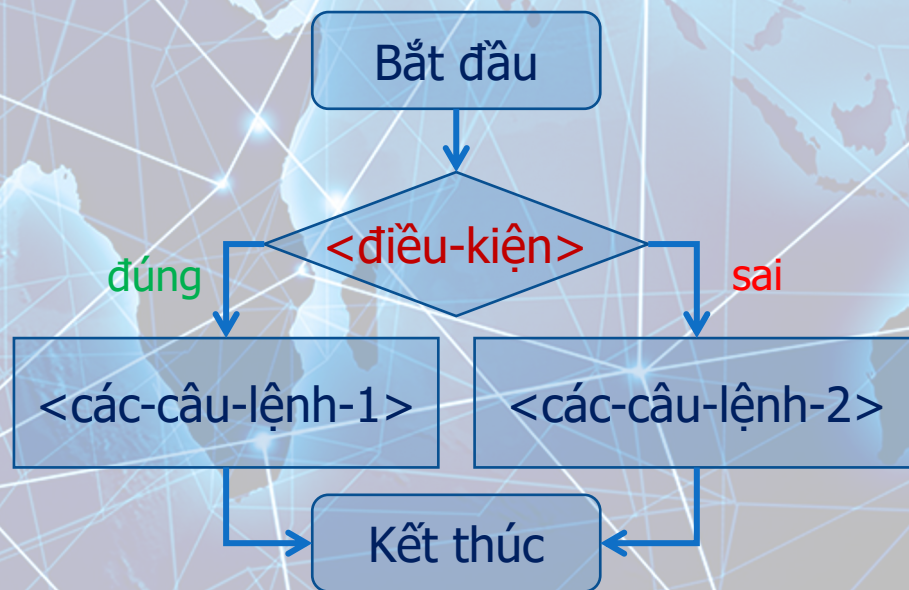
▪ Lệnh **if**

```
if (<điều-kiện>) {  
    <các-câu-lệnh>;  
}
```



▪ Lệnh **if/else** hai chiều

```
if (<điều-kiện>) {  
    <các-câu-lệnh-1>;  
} else {  
    <các-câu-lệnh-2>;  
}
```





Lệnh rẽ nhánh



- Lệnh **if/else** nhiều chiều

```
if (<điều-kiện-1>) {  
    <các-câu-lệnh-1>;  
} else if (<điều-kiện-2>) {  
    <các-câu-lệnh-2>;  
} ... else if (<điều-kiện-n>) {  
    <các-câu-lệnh-n>  
} else {  
    <các-câu-lệnh-default>;  
}
```

- Các lệnh **if** và **if/else** có thể lồng vô hạn vào nhau.



Lệnh rẽ nhánh

▪ Lệnh **switch..case**

```
switch (<bt>) {  
  case <gt-1>:  
    <khởi-lệnh-1>;  
    break;  
  case <gt-2>:  
    <khởi-lệnh-2>;  
    break;  
  ...  
  case <gt-n>:  
    <khởi-lệnh-n>;  
    break;  
  default:  
    <khởi-lệnh-default>;  
}
```



```
if (<bt> == <gt-1>) {  
  <khởi-lệnh-1>;  
} else if (<bt> == <gt-2>) {  
  <khởi-lệnh-2>;  
} ... else if (<bt> == <gt-n>) {  
  <khởi-lệnh-n>;  
} else {  
  <khởi-lệnh-default>;  
}
```




Lệnh rẽ nhánh



- Biểu thức điều kiện

`<điều-kiện> ? <biểu-thức-1> : <biểu-thức-2>`

- Nếu biểu thức `<điều-kiện>` là true thì `<biểu-thức-1>` được dùng, ngược lại thì `<biểu-thức-2>` được dùng.

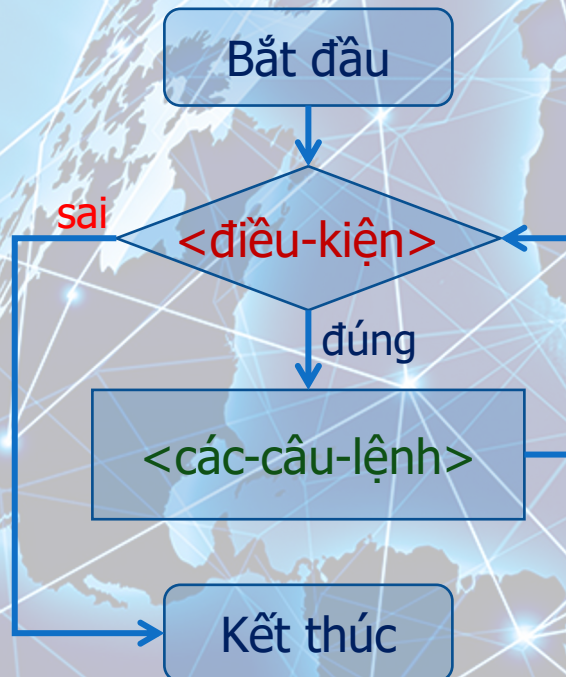
```
var a = 10;  
var b = 12;  
var max = a > b ? a : b;  
document.writeln(max); // 12
```




Lệnh lặp

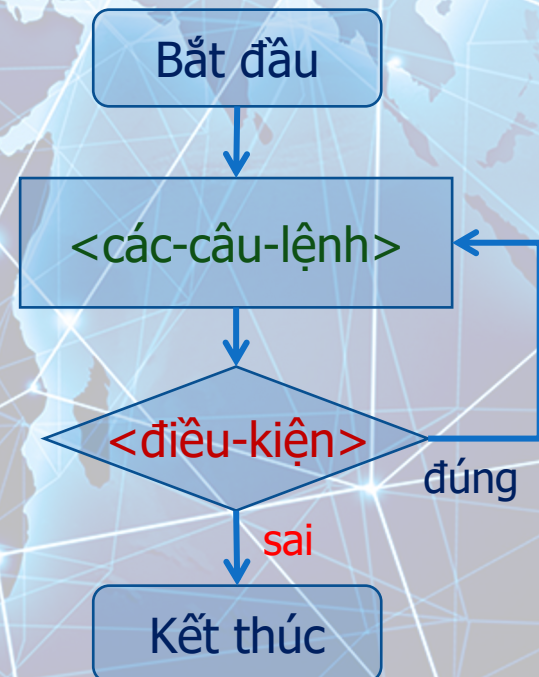
▪ Lệnh **while**

```
while (<điều-kiện>) {  
    <các-câu-lệnh>;  
}
```



▪ Lệnh **do...while**

```
do {  
    <các-câu-lệnh>;  
} while (<điều-kiện>);
```

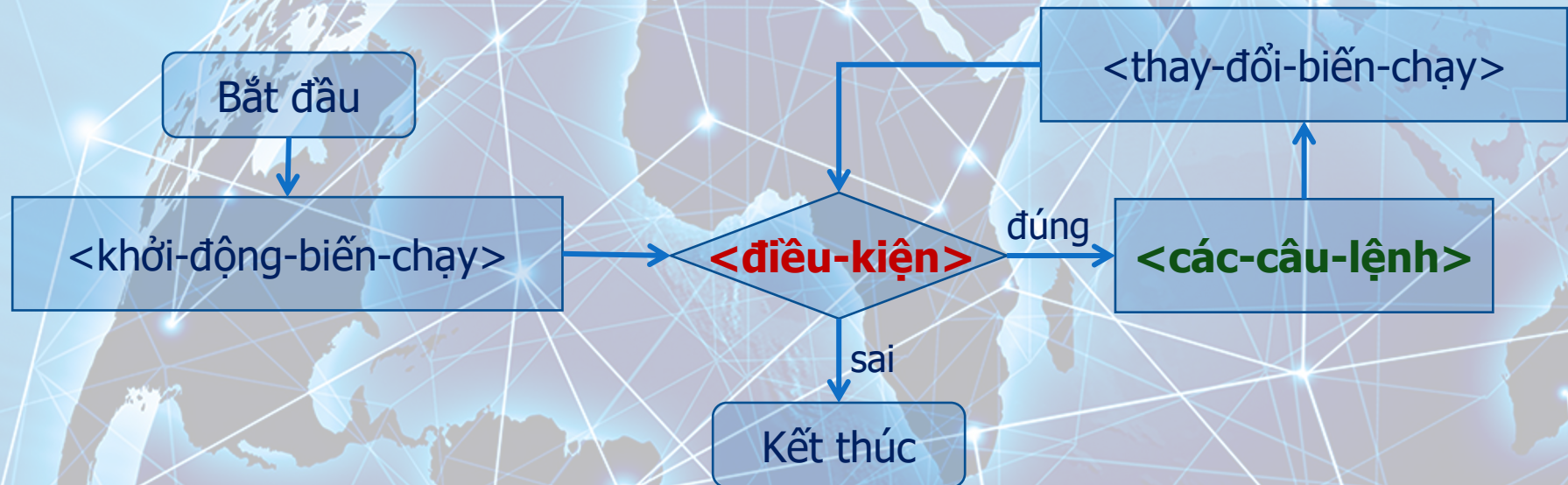




Lệnh lặp

▪ Lệnh **for**

```
for (<khởi-động-biến-chạy>; <điều-kiện>; <thay-đổi-biến-chạy>) {  
    <các-câu-lệnh>;  
}
```





Lệnh lặp

- Lệnh **for..in** được sử dụng lặp qua các thuộc tính của đối tượng.

```
var student = {  
  firstName: "Le",  
  lastName: "Duong",  
  dateOfBirth: "1995-10-10"  
}  
  
for (var s in student) {  
  document.write(s + ": " + student[s] + "<br />");  
}
```

```
firstName: Le  
lastName: Duong  
dateOfBirth: 1995-10-10
```




Lệnh break và continue

- Lệnh **break** dùng để dừng vòng lặp chứa nó.
- Lệnh **continue** dùng bỏ qua lượt lặp hiện tại và thực thi lượt lặp tiếp sau.
- Chú ý:
 - Lệnh **continue** chỉ được dùng trong các lệnh lặp.
 - Lệnh **break** ngoài dùng trong các lệnh lặp, có thể dùng trong lệnh **switch** để thoát khỏi **switch**.

JAVASCRIPT

HÀM



Định nghĩa hàm



- Để định nghĩa hàm trong javascript sử dụng từ khóa **function**, cú pháp như sau:

```
function <tên-hàm> ( [<các-tham-số>] ) {  
    [<các-câu-lệnh>];  
}
```

- Một hàm có thể **không có, hoặc có nhiều tham số** cách nhau bằng dấu phẩy (,).



Gọi hàm

- Để gọi sử dụng hàm **dùng tên hàm** và **truyền danh sách đối số** tương ứng vào hàm.
- Ví dụ

```
function xuatTong(a, b) {  
    var s = "<b>a + b</b> = " + (a + b);  
    document.write(s);  
}
```

```
xuatTong(10, 11);
```

a + b = 21

- Khi gọi hàm nếu các tham số không được truyền sẽ nhận giá trị **undefined**.



Gọi hàm



- Phân biệt hai khái niệm:
 - **Tham số** (parameters) là danh sách các tên tham số trong định nghĩa hàm.
 - **Đôi số** (arguments) là những giá trị thực sự được truyền vào hàm.
- Chú ý:
 - Không cần chỉ định **kiểu tham số**.
 - Javascript không kiểm tra **kiểu đôi số** truyền vào.
 - Javascript không kiểm tra **số đôi số** truyền vào.



Sử dụng return trong hàm

- Muốn trả về một giá trị cho hàm sử dụng từ khóa **return**, câu lệnh này thường **nằm ở cuối hàm**.
- Ví dụ hàm tìm ước chung lớn nhất hai số nguyên dương.

```
function ucln(a, b) {  
    while (a != b) {  
        if (a > b)  
            a -= b;  
        else  
            b -= a;  
    }  
  
    return a;  
}
```

```
var u = ucln(85, 150);  
document.write(u);
```




Function Literal



- Javascript 1.2 giới thiệu khái niệm **function literal** để định nghĩa một hàm, cú pháp:

```
var <tên-biến> = function ([các-tham-số]) {  
    [<các-câu-lệnh>];  
}
```

- Ví dụ

```
var f = function(x, y) {  
    return x + y;  
}  
  
var z = f(15, 20);
```



Truyền đối số vào hàm

- Truyền trị (pass by value): truyền giá trị, không phải vị trí của đối số.
- Truyền tham chiếu (pass by reference): các đối tượng được truyền tham chiếu, tức là vị trí của đối số.

JAVASCRIPT

XỬ LÝ SỰ KIỆN



Sự kiện

- Tương tác của javascript và HTML thông qua các sự kiện khi **người dùng hoặc trình duyệt có tác động đến trang web.**
- Mỗi thành phần HTML sẽ có một danh sách các sự kiện được xử lý bởi javascript.



Xử lý sự kiện



- Để xử lý sự kiện cho một thành phần HTML có thể thực hiện theo cú pháp sau:

```
<tên-thẻ tên-sự-kiện="mã-nguồn-javascript">
```

```
<tên-thẻ tên-sự-kiện='mã-nguồn-javascript'>
```

- Ví dụ hiển thị một alert box khi người dùng click trên một button.

```
<input type="button" value="Click me!"  
onclick="alert('Welcome to Javascript.')" />
```



Xử lý sự kiện

```
<script type="text/javascript">  
function sayHello() {  
    var name = document.getElementById("name").value;  
    var s = "Hello " + name;  
    alert(s);  
}  
</script>
```

```
<input type="text" id="name" placeholder="Put your  
name..." />  
<input type="button" value="Say" onclick="sayHello();" />
```





Một số sự kiện thường dùng

- **onchange**: thành phần HTML bị thay đổi.
- **onclick**: người dùng click trên một thành phần HTML.
- **onmouseover**: người dùng di chuyển chuột trên thành phần HTML.
- **onmouseout**: người dùng di chuyển chuột ra ngoài thành phần HTML.
- **onkeydown**: người dùng ấn phím trên bàn phím.
- **onload**: trình duyệt hoàn tất việc nạp trang.



Một số sự kiện thường dùng

```
<script type="text/javascript">  
function change() {  
    var name = document.getElementById("name");  
    var result = document.getElementById("result");  
    result.innerText = name.value;  
}  
</script>
```

```
<input type="text" id="name" placeholder="Put your  
name..." onchange="change();" />  
<p id="result"></p>
```

→ Nhập xong ấn enter.

JAVASCRIPT

ĐỐI TƯỢNG



Đối tượng

- Javascript là ngôn ngữ lập trình hướng đối tượng.
- Các đối tượng có sẵn trong javascript hoặc do người dùng định nghĩa đều là con của đối tượng kiểu **Object**.



Đối tượng

- Các đối tượng trong javascript cũng là các biến với nhiều giá trị các nhau bằng dấu phẩy (,), mỗi giá trị là cặp **name:value**.
- Ví dụ:

```
var student = {  
  firstName: "Le",  
  lastName: "Duong",  
  dateOfBirth: "1995-10-10",  
  location: "Tp.HCM",  
  gender: "Male"  
}
```

name

value

firstName: "Le"



Thuộc tính và phương thức

- Mỗi cặp **name:value** trong đối tượng javascript gọi là các **thuộc tính**.
- Phương thức là **thể hiện hành động** mà đối tượng có thể thực hiện, cú pháp khái báo phương thức:

```
<tên-phương-thức>: function() {  
    [<các-câu-lệnh>];  
}
```




Thuộc tính và phương thức

- Ví dụ

```
var student = {  
  firstName: "Le",  
  lastName: "Duong",  
  dateOfBirth: "1995-10-10",  
  location: "Tp.HCM",  
  gender: "Male",  
  fullName: function() {  
    return this.firstName + this.lastName;  
  }  
}
```



Truy cập thuộc tính

- Truy cập vào thuộc tính của đối tượng theo một trong hai cách sau:

```
<tên-đối-tượng>.<tên-thuộc-tính>
```

```
<tên-đối-tượng>["<tên-thuộc-tính>"]
```

- Ví dụ

```
document.writeln(student.firstName);  
document.writeln(student["firstName"]);
```




Truy cập phương thức

- Để truy cập vào phương thức sử dụng cú pháp:

```
<tên-đối-tượng>.<tên-phương-thức>([tham-số]);
```

- Ví dụ:

```
document.writeln(student.fullName());
```



Toán tử new



- Toán tử **new** được sử dụng để tạo ra một **thể hiện** của đối tượng.
- Để tạo một đối tượng sử dụng toán tử **new**, kết hợp với một phương thức tạo theo sau.
- Ví dụ tạo đối tượng Object

```
<script type="text/javascript">  
var user = new Object();  
user.firstName = "Thanh";  
user.lastName = "Duong";  
</script>
```




Toán tử new

- Ví dụ tạo đối tượng sử dụng hàm. Từ khóa **this** tham chiếu đến đối tượng được truyền vào hàm.

```
<script type="text/javascript">
function user(fName, lName) {
    this.firstName = fName; // thuộc tính
    this.lastName = lName; // thuộc tính
    this.fullName = function() { // phương thức
        return this.firstName + " " + this.lastName;
    }
}

var u = new user("Thanh", "Duong");
document.write("<b>Full name: </b>" + u.fullName());
</script>
```



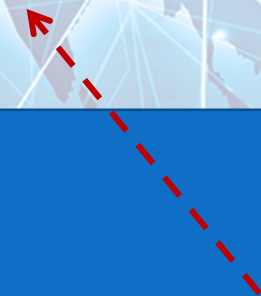
Từ khóa with

- Từ khóa **with** được dùng truy cập các thuộc tính và phương thức của đối tượng một cách ngắn gọn, mà không cần thông qua tên đối tượng và toán tử "."

Không cần **u.firstName**, **u.fullName()**

- Ví dụ

```
<script type="text/javascript">
var u = new user("Thanh", "Duong");
with (u) {
    document.write("<b>First name: </b>" + firstName);
    document.write("<b>Full name: </b>" + fullName());
}
</script>
```



JAVASCRIPT

The background of the slide features a stylized world map in a light blue and purple hue. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The lines are thin and create a web-like pattern across the entire map.

NUMBER



Number

- Trong javascript chỉ có một loại số, số có thể được viết dạng số nguyên hoặc dạng thập phân (decimal) hoặc ký hiệu mũ.

- Ví dụ:

```
var x = 1.0; // dạng thập phân  
var y = 1; // dạng nguyên  
var z = 15e6; // 15000000  
var f = 15e-6; // 0.000015
```

- Số có thể là một giá trị cơ bản hoặc một đối tượng

```
var x = 10; // typeof x → "number"  
var y = new Number(10); // typeof y → "object"
```




Number

- Trong javascript, số luôn là **số chấm động 64 bit** định dạng chuẩn IEEE 754 (có giá trị $-(2^{53} - 1)$ đến $2^{53} - 1$), trong đó 52 bit đầu biểu diễn phần số, 11 bit tiếp theo biểu diễn phần mũ, 1 bit cuối biểu diễn dấu.
- Kiểu số có 3 giá trị đặc biệt
 - Infinity: số dương vô cực ($1/+0 \rightarrow +\text{Infinity}$)
 - -Infinity: số âm vô cực ($1/-0 \rightarrow -\text{Infinity}$)
 - NaN: giá trị không phải số ($100/"a" \rightarrow \text{NaN}$)



Các thuộc tính



- `Number.MAX_VALUE`: số lớn nhất.
- `Number.MIN_VALUE`: số nhỏ nhất.
- `Number.NEGATIVE_INFINITY`: âm vô cực.
- `Number.POSITIVE_INFINITY`: dương vô cực.
- `Number.NaN`: giá trị không phải số. (Not a Number)



Các phương thức



- **isNaN()**: kiểm tra một giá trị có phải là số không.
- **isInteger()**: kiểm tra một giá trị có phải số nguyên không.
- **isFinite()**: kiểm tra một giá trị có phải hữu hạn không.
- **isSafeInteger()**: kiểm tra một giá trị có phải số nguyên an toàn không.



Các phương thức



- **toString()**: chuyển số thành chuỗi.
- **toExponential()**: trả về một chuỗi được làm tròn và được ghi theo ký hiệu mũ.
- **toFixed()**: trả về một chuỗi với số chữ số thập được chỉ định, mặc định là 0.
- **toPrecision()**: trả về một chuỗi số ghi theo chiều dài chỉ định.
- **valueOf()**: chuyển đổi tượng Number sang kiểu số cơ bản.



Các phương thức

▪ Ví dụ

```
var x = 1.12646;  
console.info(x.toExponential()); // 1.12646e+0  
console.info(x.toExponential(2)); // 1.13e+0  
  
console.info(x.toFixed()); // 1  
console.info(x.toFixed(2)); // 1.13  
  
console.info(x.toPrecision()); // 1.12646  
console.info(x.toPrecision(5)); // 1.1265  
  
console.info(x.toString()); // 1.12646  
console.info(x.toString(2)); // 10010110  
console.info(x.toString(8)); // 226  
console.info(x.toString(16)); // 96
```



Chuyển các biến thành các số



- Javascript cung cấp 3 phương thức toàn cục để chuyển các biến thành các số.
 - **Number()**: trả về số.
 - **parseFloat()**: trả về số chấm động.
 - **parseInt()**: trả về số nguyên.
- Khi **không thể chuyển** một biến thành số thì các phương thức này trả về giá trị NaN.



Chuyển các biến thành các số

- Ví dụ:

```
console.info(Number(true)); // 1
console.info(Number(false)); // 0
console.info(Number("9")); // 9
console.info(Number("a5")); // NaN

console.info(parseInt("10")); // 10
console.info(parseInt("9.7")); // 9

console.info(parseFloat("9.7")); // 9.7
```

JAVASCRIPT

The background of the slide features a stylized world map in shades of blue and purple. Overlaid on the map is a complex network of white lines connecting various glowing white nodes, creating a global connectivity theme.

MATH



Math

- Math cung cấp các thuộc tính và phương thức toán học.
- Các phương thức và thuộc tính của Math đều là tĩnh (static).



Các phương thức thông dụng

- **abs()**: lấy trị tuyệt đối của một số.
- **sqrt()**: tính căn bậc hai của một số.
- **max()**: trả về số lớn nhất của các số.
- **min()**: trả về số nhỏ nhất của các số.
- **random()**: trả về một số ngẫu nhiên từ 0 đến 1.

```
document.writeln(Math.abs(-10)); // 10  
document.writeln(Math.sqrt(144)); // 12  
document.writeln(Math.max(5, 3, 7, 9, 4)); // 9  
document.writeln(Math.min(5, 3, 7, 2, 9)); // 2  
document.writeln(Math.random());
```




Các phương thức thông dụng

- **ceil()**: lấy số nguyên nhỏ nhất còn lớn hơn hoặc bằng số đang xét.
- **floor()**: lấy số nguyên lớn nhất còn nhỏ hơn hoặc bằng số đang xét.
- **round()**: trả về số nguyên được làm tròn gần nhất với số đang xét.

```
document.writeln(Math.ceil(4.2)); // 5  
document.writeln(Math.ceil(-4.7)); // -4  
document.writeln(Math.floor(4.7)); // 4  
document.writeln(Math.floor(-4.2)); // -5  
document.writeln(Math.round(4.5)); // 5  
document.writeln(Math.round(-4.2)); // -4
```



Các phương thức thông dụng

- **pow()**: tính số mũ.
- **exp()**: tính e^n .
- **log()**: tính $\ln x$.
- **sin()**, **cos()**, **tan()**: tính sin, cos, tan.
- **asin()**, **acos()**, **atan()**: hàm ngược của sin, cos, tan trả về giá trị radian.

```
document.writeln(Math.pow(4, 2)); // 16
document.writeln(Math.log(5)); // 1.6094379124341003
document.writeln(Math.exp(2)); // 7.38905609893065
document.writeln(Math.sin(Math.PI/6));
document.writeln(Math.cos(Math.PI/3));
```


JAVASCRIPT

MẢNG



Mảng

- Mảng là một biến lưu trữ nhiều giá trị cùng lúc.
- Trong javascript, các phần tử trong mảng có thể có **kiểu dữ liệu khác nhau**.
- Truy cập một phần tử mảng dùng chỉ số mảng.



Mảng

- Tạo mảng theo một trong 2 ví dụ sau:

```
var colors = new Array("red", "green", "blue");  
var animals = ["dog", "cat", "tiger"];
```

- Trong đó cách thứ 2 được ưa dùng hơn. Việc sử dụng **new Array** sẽ dẫn đến một số nhập nhằng khi viết code, chẳng hạn:

```
var a = new Array(30, 20);  
document.write(a[0]); // 30  
  
var b = new Array(30);  
document.write(b[0]); // undefined
```

Định nghĩa mảng có 2 phần tử là 30 và 20

Định nghĩa mảng có 30 phần tử đều có giá trị undefined.



Mảng

- Ví dụ truy xuất phần tử trong mảng

```
<script type="text/javascript">
var animals = ["dog", "cat", "tiger"];

// Chiều dài mảng
document.writeln(animals.length); // 3

// Truy cập vào các phần tử mảng
for (var i = 0; i < animals.length; i++) {
    document.writeln(animals[i]);
}
</script>
```




Các thuộc tính

- **length**: lấy chiều dài mảng.
- **prototype**: dùng thêm thuộc tính và phương thức vào đối tượng.

```
// Thêm phương thức mới vào các đối tượng Array
Array.prototype.isDigitArray = function() {
    var isDigit = true;
    for (var i = 0; i < this.length && isDigit; i++)
        if (typeof this[i] !== "number")
            isDigit = false;

    return isDigit;
};
var a = [5, 6, 3, 8, 5];
console.info(a.length); // 5
console.info(a.isDigitArray()); // true
```



Các phương thức thông dụng

- **push()**: thêm một hoặc nhiều phần tử ở **cuối mảng** và trả về chiều dài mới của mảng.
- **pop()**: lấy phần tử ở **cuối** ra khỏi mảng.
- **unshift()**: thêm một hoặc nhiều phần tử ở **đầu mảng** và trả về chiều dài mới của mảng
- **shift()**: lấy phần tử ở **đầu** ra khỏi mảng.



Các phương thức thông dụng

▪ Ví dụ

```
var colors = ["red", "green", "blue"];  
document.writeln(colors.pop()); // blue  
colors.push("yellow", "green"); // 4  
document.writeln(colors); // red, green, yellow, green  
document.writeln(colors.shift()); // red  
colors.unshift("gray"); // 4  
document.writeln(colors); // gray, green, yellow, green
```



Các phương thức thông dụng

- **indexOf()**: trả về vị trí đầu tiên tìm thấy phần tử hoặc -1 nếu không tìm thấy.

```
array.indexOf(searchElement [, fromIndex]);
```

fromIndex là tham số tùy chọn, dùng chỉ định vị trí bắt đầu để tìm kiếm, mặc định là 0.

- **lastIndexOf()**: trả về vị trí cuối cùng tìm thấy phần tử hoặc -1 nếu không tìm thấy.

```
array.lastIndexOf(searchElement [, lastIndex]);
```

lastIndex chỉ định vị trí kết thúc tìm kiếm, mặc định là chiều dài mảng



Các phương thức thông dụng

- Ví dụ

```
var colors = ["red", "green", "blue", "green"];  
document.writeln(colors.indexOf("green")); // 1  
document.writeln(colors.indexOf("green", 2)); // 3  
document.writeln(colors.lastIndexOf("green")); // 3  
document.writeln(colors.lastIndexOf("green", 2)); // 1
```



Các phương thức thông dụng

- **reverse()**: đảo ngược mảng.
- **sort()**: sắp xếp mảng.

```
array.sort([<hàm-so-sánh>]);
```

- Ví dụ:

```
var numbers = [1, 4, 7, 3];  
numbers.reverse();  
document.writeln(numbers); // 3, 7, 4, 1  
numbers.sort(); // sắp tăng  
document.writeln(numbers); // 1, 3, 4, 7  
  
numbers = [1, 6, 7];  
numbers.sort(function(a, b) { // sắp giảm  
    return b - a;  
});  
document.writeln(numbers);
```




Các phương thức thông dụng

- **join()**: dùng kết hợp các phần tử của mảng thành một chuỗi.

```
array.join(separator);
```

- Ví dụ:

```
var colors = ["red", "green", "blue"];  
document.write(colors.join(", ")); // red, green, blue  
document.write(colors.join("; ")); // red; green; blue  
document.write(colors.join("|")); // red|green|blue
```



Các phương thức thông dụng

- **slice()**: cắt một đoạn trong mảng và trả về mảng mới.

```
array.slice(beginIndex [, endIndex]);
```

- beginIndex và endIndex: lấy đoạn con từ vị trí beginIndex đến trước endIndex, endIndex mặc định là chiều dài mảng.
- Nếu các **chỉ số là số âm** sẽ được tính từ cuối mảng.



Các phương thức thông dụng

- Ví dụ chỉ số âm

0	1	2	3	4
red	green	blue	yellow	gray
-5	-4	-3	-2	-1

- Ví dụ sử dụng slice

```
var colors = ["red", "green", "blue", "yellow", "gray"];  
document.writeln(colors.slice(2)); // blue, yellow, gray  
document.writeln(colors.slice(1, 3)); // green, blue  
document.writeln(colors.slice(-2)); // yellow, gray  
document.writeln(colors.slice(-3, -2)); // blue
```



Các phương thức thông dụng

- **splice()**: dùng thay đổi nội dung của mảng, thêm hoặc xóa các phần tử trong mảng và trả về các phần tử đã bị xóa.

```
array.splice(index, num, [e1, e2, ..., en]);
```

- index: chỉ định vị trí bắt đầu thay đổi mảng.
- num: chỉ định số lượng phần tử sẽ bị xóa từ index, mặc định là 0.
- e₁, e₂, ..., e_n chỉ định các phần tử sẽ được thêm vào từ vị trí index.



Các phương thức thông dụng

- Ví dụ sử dụng phương thức splice

```
var colors = ["red", "green", "yellow"];  
colors.splice(1, 0, "orange");  
document.writeln(colors); // red, orange, green, yellow  
colors.splice(1, 2, "gold");  
document.writeln(colors); // red, gold, yellow  
colors.splice(1, 1);  
document.writeln(colors); // red, yellow
```



Các phương thức thông dụng

- **concat()**: trả về mảng mới bao gồm các phần tử nối mảng đang xét với các mảng hoặc phần tử chỉ định.
- Ví dụ

```
var numbers = [1, 2];  
var nNumbers = numbers.concat([3, 4], [5], 8, 9);  
document.writeln(nNumbers); // 1, 2, 3, 4, 5, 8, 9
```




Các phương thức khác

- **forEach()**: lặp qua các phần tử để xử lý trong một hàm cho trước.
- Ví dụ:

```
var colors = ["red", "green", "blue"];  
colors.forEach(function(value, index) {  
    document.writeln("colors " + index + ": " + value);  
});
```



Các phương thức khác

- **map()**: trả về mảng mới (**cùng kích thước** mảng đang xét) từ kết quả xử lý mỗi phần tử mảng đang xét trong một hàm cho trước.
- Ví dụ:

```
var colors = ["red", "green", "yellow"];  
var upperColors = colors.map(function(c) {  
    return c.toUpperCase();  
});  
  
// RED, GREEN, YELLOW  
document.writeln(upperColors);
```




Các phương thức khác

- **filter()**: trả về mảng mới là **một số phần tử** trong mảng đang xét thỏa bộ lọc định nghĩa trong một hàm cho trước.
- Ví dụ

```
var numbers = [12, 5, 3, 8, 9, 26];  
var evenNum = numbers.filter(function(n) {  
    return n % 2 == 0;  
});  
document.writeln(evenNum); // 12, 8, 26
```

JAVASCRIPT

CHUỖI



Chuỗi



- Đối tượng String được sử dụng thao tác với chuỗi trong javascript.
 - Ví dụ:

```
var u1 = "Hello";  
var u2 = new String("Hello");
```
- Một chuỗi trong javascript là văn bản đặt giữa cặp dấu nháy đôi (" ") hoặc nháy đơn (' ').
 - Ví dụ: "Hello", 'Hello'
- Javascript tự động chuyển chuỗi thành một đối tượng String nên có thể sử dụng tất cả các phương thức của đối tượng Object.



Các thuộc tính chuỗi

- **length**: lấy chiều dài chuỗi.
- **prototype**: cho phép thêm các thuộc tính và phương thức cho đối tượng.

```
// Thêm phương thức vào các đối tượng String
String.prototype.wordCount = function() {
    return this.split(" ").length;
};
```

```
var s = "Open University";
console.info(s.length); // 15
console.info(s.wordCount()); // 2
```




Các phương thức chuỗi

- **charAt(<index>)**: trả về ký tự tại vị trí chỉ định.
- **charCodeAt(<index>)**: trả về mã ASCII của ký tự tại vị trí chỉ định.

```
var s = "Javascript is great.";
var s1 = s.charAt(1); // a
var s2 = s[1]; // a
var a1 = s.charCodeAt(1); // 97
```



Các phương thức chuỗi

- **toLowerCase()**: chuyển chuỗi thành các ký tự thường.
- **toUpperCase()**: chuyển chuỗi thành các ký tự hoa.
- **concat(<string>)**: nối hai chuỗi thành chuỗi mới.

```
var s = "Javascript is great.";
var s1 = s.toLowerCase();
var s2 = s.toUpperCase();
var s3 = s.concat(" Javascript is simple to learn.");
```




Các phương thức chuỗi



- **indexOf(s [, fromIdx]):** trả về vị trí đầu tiên tìm thấy chuỗi s trong chuỗi đang xét tính từ fromIdx (mặc định 0), và -1 nếu không tìm thấy.
- **lastIndexOf(s [, endIdx]):** trả về vị trí cuối cùng tìm thấy chuỗi s trong chuỗi đang xét tính từ đầu đến endIdx - 1 (mặc định endIdx là chiều dài chuỗi), và -1 nếu không tìm thấy.

```
var s = "Javascript is great. Javascript is simple to learn.";
document.writeln(s.indexOf("Javascript")); // 0
document.writeln(s.indexOf("Javascript", 15)); // 21
document.writeln(s.lastIndexOf("Javascript")); // 21
document.writeln(s.lastIndexOf("Javascript", 20)); // 0
```



Các phương thức chuỗi

- **substring(start [, end])**: trả về chuỗi con từ start đến end - 1, mặc định end là chiều dài chuỗi.

```
var s = "Javascript is great";  
document.writeln(s.substring(11, 13)); // is  
document.writeln(s.substring(14)); // great
```

- Chú ý: start và end **không thể nhận giá trị âm**.



Các phương thức chuỗi

- **substr(start [, length])**: trả về chuỗi con có length ký tự tính từ vị trí start.
 - Nếu không chỉ định length thì chuỗi con sẽ tính từ start đến cuối chuỗi.
 - Nếu start < 0 sẽ được tính từ cuối chuỗi.

```
var s = "Javascript is great";  
document.writeln(s.substr(11)); // is great  
document.writeln(s.substr(11, 2)); // is  
document.writeln(s.substr(-5)); // great  
document.writeln(s.substr(-8, 2)); // is
```



Các phương thức chuỗi

- **slice(start [, end])**: trả về chuỗi con mới từ start đến end - 1, nếu start hoặc end là số âm sẽ tính từ cuối chuỗi.

```
var s = "Javascript-is-great";  
document.writeln(s.slice(11, 13)); // is  
document.writeln(s.slice(-8, 13)); // is  
document.writeln(s.slice(-8, -6)); // is  
document.writeln(s.slice(14)); // great  
document.writeln(s.slice(-5)); // great
```




Các phương thức chuỗi

- **split(seperator [, limit])**: tách chuỗi thành một mảng các chuỗi con với dấu hiệu tách là seperator, limit là số nguyên giới hạn số lượng chuỗi con tách.

```
var s = "Javascript-is-great";  
document.writeln(s.split("-")); // Javascript,is,great  
document.writeln(s.split("-", 2)); // Javascript,is
```



Các phương thức chuỗi

- **replace(oldString, newString)**: thay thế chuỗi con oldString trong chuỗi đang xét thành chuỗi newString.
- **search()**: trả về chỉ số của chuỗi con khớp với một biểu thức chính quy, và -1 nếu không tìm thấy.

```
var s = "Javascript is great.";

// Javascript is good.
var s1 = s.replace("great", "good");

// 14
var a1 = s.search("great");
```


JAVASCRIPT

The background of the slide features a stylized world map in a light blue and purple hue. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The lines are thin and create a web-like pattern across the entire map.

DATE



Date

- Đối tượng Date được dùng thao tác dữ liệu ngày giờ (năm, tháng, ngày, giờ, phút, giây, mili giây).
- Tạo đối tượng Date thông qua 4 hàm tạo sau
 - `new Date();` → trả về thời điểm hiện tại
 - `new Date(<milliseconds>);`
 - `new Date(<date-string>);`
 - `new Date(<year>, <month>, <day>, <hours>, <minutes>, <seconds>, <milliseconds>);`



Các định dạng Date

- Có 4 loại định dạng ngày tháng trong javascript
 - **ISO Date** (ISO 8601 - chuẩn quốc tế): 2017-05-27
 - **Short Date** (MM/DD/YYYY): 05/27/2017
 - **Long Date** (MMM DD YYYY):
 - Chuỗi định dạng không phân biệt ký tự hoa, thường.
 - Thứ tự ngày, tháng, năm có thể thay đổi.
 - Tháng có thể viết tắt hoặc viết đầy đủ.
 - Có thể sử dụng dấu phẩy (,) để phân cách
 - Ví dụ: "Jun 27 2017", "JUN 27 2017", "27 June 2017", "2017 June 27"
 - **Full Date**: "Saturday May 27 2017"



Các định dạng Date

- Ví dụ định dạng ISO

```
// YYYY-MM-DD
var d1 = new Date("2017-05-27");
// YYYY-MM
var d2 = new Date("2017-05");
// YYYY
var d3 = new Date("2017");
// YYYY-MM-DDTHH:MM:SSZ
var d4 = new Date("2017-05-27T10:30:05Z");
```

- Chú ý: YYYY-MM-DDTHH-MM-SSZ nếu bỏ qua các ký tự T và Z thì kết quả có thể khác nhau trên từng trình duyệt.



- Khi thiết lập (set) date mà không chỉ định timezone thì javascript sẽ lấy timezone của trình duyệt.
- Khi lấy (get) date mà không chỉ định timezone thì javascript sẽ tự động chuyển sang timezone của trình duyệt.



Các phương thức thông dụng

- **getDate()**: lấy ngày của tháng (1 – 31)
- **getDay()**: lấy ngày trong tuần (0 – 6) (0: chủ nhật, 1-6: thứ 2 → thứ 7)
- **getFullYear()**: lấy năm dạng 4 số (yyyy)
- **getHours()**: lấy giờ (0 – 23)
- **getMinutes()**: lấy phút (0 – 59)
- **getSeconds()**: lấy giây (0 – 59)
- **getMilliseconds()**: lấy mili giây (0 – 999)
- **getMonth()**: lấy tháng (0 – 11: tháng 1 → tháng 12)
- **getTime()**: lấy thời gian (mili giây từ 1/1/1970)



Các phương thức thông dụng

- **setDate()**: thiết lập ngày của tháng.
- **setFullYear()**: thiết lập năm.
- **setHours()**: thiết lập giờ.
- **setMilliseconds()**: thiết lập mili giây
- **setMinutes()**: thiết lập phút.
- **setMonth()**: thiết lập tháng.
- **setSeconds()**: thiết lập giây.
- **setTime()**: thiết lập thời gian (mili giây từ 1/1/1970).



Các phương thức thông dụng

- **Date.parse()**: trả về số mili giây (tính từ 1/1/1970) của một chuỗi ngày tháng hợp lệ.

```
var x = new Date();  
var y = new Date();  
var z = new Date();  
// Lấy ngày trước ngày hiện tại  
x.setDate(x.getDate() - 1);  
// lấy ngày sau ngày hiện tại  
y.setDate(y.getDate() + 1);  
// Xác định thứ của ngày này năm sau  
console.info(z.getDay()); // 0 - chủ nhật  
z.setFullYear(z.getFullYear() + 1);  
console.info(z.getDay()); // 1 - thứ hai  
  
var d = Date.parse("May 28 2017"); // 1495904400000
```




Các phương thức thông dụng

- Các phương thức cho UTC (Universal Time Zone)
 - `getUTCDate()`
 - `getUTCDay()`
 - `getUTCFullYear()`
 - `getUTCHours()`
 - `getUTCMilliseconds()`
 - `getUTCMinutes()`
 - `getUTCMonth()`
 - `getUTCSeconds()`

JAVASCRIPT

The background of the slide features a stylized world map in a light blue hue. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The overall aesthetic is modern and technological.

HTML DOM



HTML DOM



- HTML DOM (Document Object Model) là mô hình đối tượng chuẩn và giao diện (interface) lập trình cho HTML.
- HTML DOM định nghĩa
 - Các thành phần (**elements**) HTML (là các object).
 - Thuộc tính (**attributes**) các thành phần HTML.
 - Các phương thức (**methods**) truy cập vào các thành phần HTML.
 - Các sự kiện (**events**) cho các thành phần HTML.



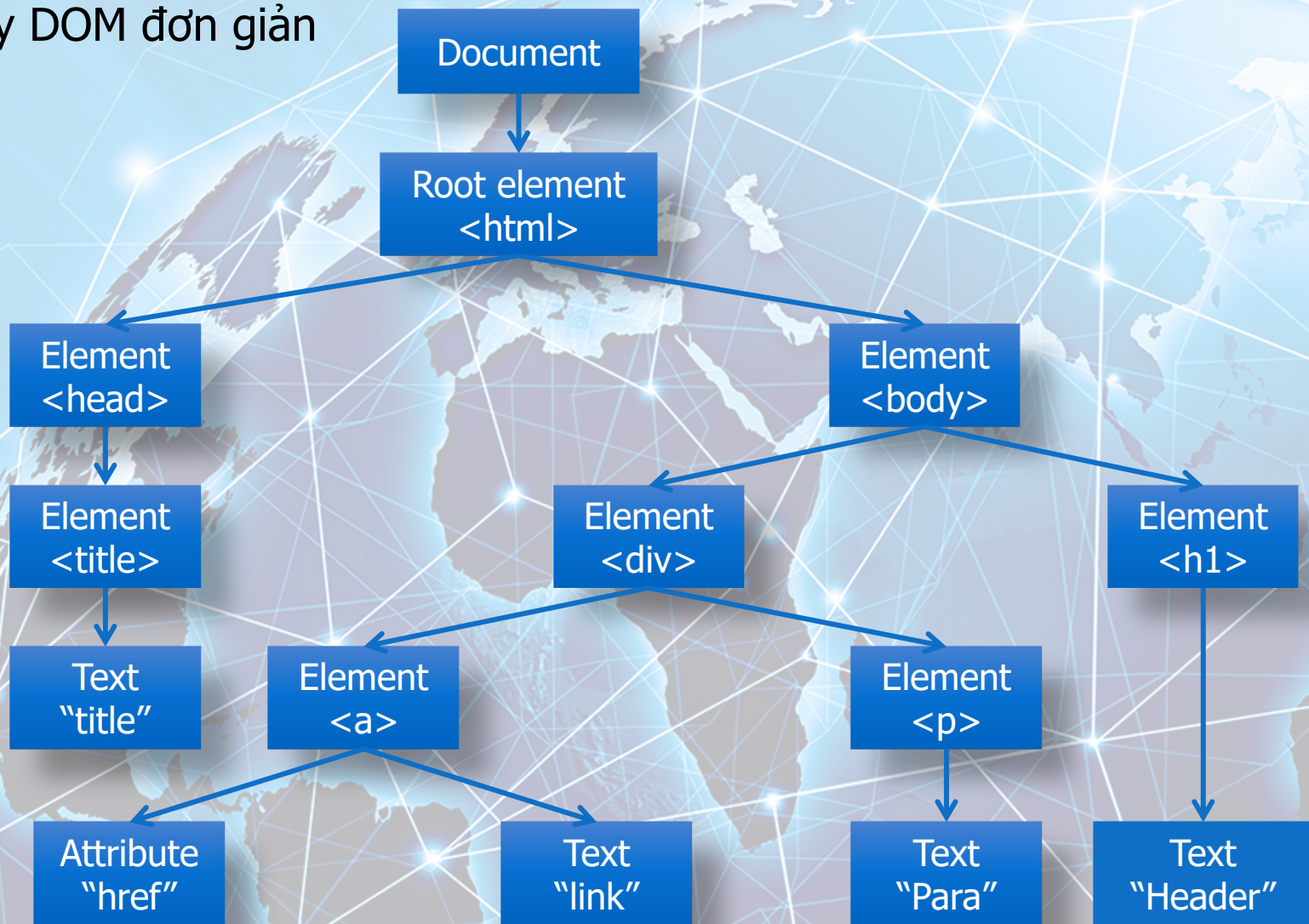
HTML DOM

- Javascript **có thể thay đổi tất cả** các thành phần của tài liệu HTML thông qua DOM.
- Khi trang web được nạp, trình duyệt sẽ tạo DOM cho trang.
- Mô hình HTML DOM được xây dựng dạng cây các Object.



HTML DOM

Ví dụ cây DOM đơn giản





Đối tượng document

- Đối tượng document **chứa tất cả** các đối tượng khác trong trang web.
- Truy cập vào bất cứ thành phần trên trang HTML nào cũng thông qua đối tượng document.



Tìm thành phần



- Tìm theo thuộc tính **id** của thành phần.
- Tìm theo **tên** thành phần.
- Tìm theo giá trị thuộc tính **name** của thành phần.
- Tìm theo giá trị thuộc tính **class** của thành phần.
- Tìm theo **CSS selector**.



Tìm thành phần



- **document.getElementById**: tìm dựa trên giá trị thuộc tính id của các thành phần.
 - Ví dụ: `document.getElementById("username")`
 - Tìm thành phần có thuộc tính id = "username"
 - Nếu không tìm thấy, hàm sẽ trả về **undefined**.
- **document.getElementsByTagName**: tìm dựa trên tên thành phần.
 - Ví dụ: `document.getElementsByTagName("div")`
 - Tìm các thành phần `<div>`



Tìm thành phần



- Ví dụ: cho HTML như sau

```
<input id="firstNameId" type="text"
      placeholder="First name" />
<input id="lastNameId" type="text"
      placeholder="Last name" />
<input type="button" value="Show"
      onclick="show();" />
```

- Yêu cầu: Khi click button Show sẽ lấy thông tin first name và last name từ hai input để hiển thị trên alert dạng "Hello first_name last_name".

First name	Last name	Show
------------	-----------	------



Tìm thành phần

- Xử lý javascript

```
function show() {  
    var fn = document.getElementById("firstNameId");  
    var ln = document.getElementById("lastNameId");  
    if (fn != undefined && ln != undefined) {  
        var fullName = fn.value + " " + ln.value;  
        alert("Hello " + fullName)  
    }  
}
```




Tìm thành phần



- Ví dụ: kết hợp tìm theo id và tên thành phần
 - Tìm các thành phần `<p>` bên trong thành phần có id là "main"

```
var m = document.getElementById("main");  
var p = m.getElementsByTagName("p");
```




Tìm thành phần



- **document.getElementsByClassName**: tìm dựa trên giá trị thuộc tính class.
 - `document.getElementsByClassName("success")`
 - Tìm các thành phần có thuộc tính class chứa giá trị "success".
- **document.getElementsByName**: tìm dựa trên giá trị thuộc tính name.



Tìm thành phần



- Sử dụng selector giống CSS để tìm tất cả các thành khớp với selector được chỉ định dùng phương thức **querySelectorAll**.
- Ví dụ:

<code>document.querySelectorAll("p.data");</code>	Tìm tất cả các thành phần p có class là data.
<code>document.querySelectorAll("div > p");</code>	Tìm các thành phần p là con trực tiếp của thành phần div
<code>document.querySelectorAll("#main div");</code>	Tìm các thành phần div là con thành phần có id là main.



Thay đổi thành phần HTML



- Gọi e là một thành phần HTML, dùng các thuộc tính sau để thay đổi thành phần e.
- **Thay đổi nội dung** thành phần HTML
 - `e.innerHTML = <new-html>;`
- **Thay đổi giá trị thuộc tính** thành phần HTML
 - `e.<attribute> = <value>;`
 - `e.setAttribute(<attribute>, <value>);`
- **Thay đổi trình bày css** của thành phần HTML
 - `e.style.<property> = <new-style>;`



Thay đổi thành phần HTML



- Ví dụ: cho HTML như sau

```
<div class="html">HTML</div>
<div class="css">CSS</div>
<div class="html">HTML5</div>
<div class="css">CSS3</div>
<input type="button" value="HTML"
      onclick="change('html')">
<input type="button" value="CSS"
      onclick="change('css')">
```

HTML
CSS
HTML5
CSS3

HTML

CSS

- Yêu cầu: khi click trên button HTML và CSS lần lượt sẽ thay đổi style của các thẻ div tương ứng với class chứa html và css.



Thay đổi thành phần HTML

```
function change(className) {  
    var e = document.getElementsByClassName(className);  
    for (var i = 0; i < e.length; i++) {  
        e[i].innerHTML = "<h1>" + e[i].innerText + "</h1>";  
        e[i].style.color = "white";  
        switch (className) {  
            case "html":  
                e[i].style.backgroundColor = "blue";  
                break;  
            case "css":  
                e[i].style.backgroundColor = "brown";  
            }  
        }  
    }  
}
```

Xử lý javascript

- HTML
 - CSS
 - HTML5
 - CSS3
- HTML

CSS

HTML

CSS

HTML5

CSS3

- HTML

CSS

HTML

CSS

HTML5

CSS3

- HTML

CSS



Thêm sự kiện



- Trong phần xử lý sự kiện ta đã biết cách thêm xử lý sự kiện thông qua **các thuộc tính sự kiện** của các thành phần như onclick, onchange, ...
- Trong phần này ta sẽ dùng HTML DOM để xử lý sự kiện.
 - `element.<event> = function() {};`



Thêm sự kiện

- Ví dụ: cho HTML và CSS như sau

```
<button id="hover" class="btn warning">Hover me!</button>  
<button id="click" class="btn success">Click me!</button>
```

```
.btn {  
  border: none;  
  padding: 20px 20px;  
  color: white;  
  cursor: pointer;  
}
```

```
.warning {  
  background-color: #ff9800;  
}  
.warning:hover {  
  background-color: #e68a00;  
}  
.success {  
  background-color: #4CAF50;  
}  
.success:hover {  
  background-color: #46a049;  
}
```

Hover me!

Click me!



Thêm sự kiện

```
var e1 = document.getElementById("hover");
if (e1 != undefined) {
    e1.onmouseover = function() {
        e1.innerHTML = "Thank you";
    }
    e1.onmouseleave = function() {
        e1.innerHTML = "Hover me!";
    }
}

var e2 = document.getElementById("click");
if (e2 != undefined) {
    e2.onclick = function() {
        alert("Thank you");
    }
}
```

Xử lý sự kiện dùng HTML DOM



Lắng nghe sự kiện



- Ta **có thể thêm nhiều trình xử lý sự kiện** (event handler), thậm chí là hai sự kiện giống nhau trên cùng một thành phần bằng cách dùng phương thức **`addEventListener()`**.

```
element.addEventListener(event, function, useCapture)
```

- **event**: loại sự kiện xử lý như click, mousedown, mouseup, ...
- **function**: hàm được gọi khi sự kiện xảy ra.
- **useCapture**: chỉ định cách lan truyền sự kiện là capturing hay bubbling, mặc định là false.



Lắng nghe sự kiện



▪ Ví dụ:

```
<button id="click">Click me!</button>  
<p id="result"></p>
```

```
var r = document.getElementById("result");  
var e = document.getElementById("click");  
e.addEventListener('click', function() {  
    if (r.innerText == '')  
        r.innerText = "Great!!!";  
    else  
        r.innerText = "";  
});
```




Lắng nghe sự kiện



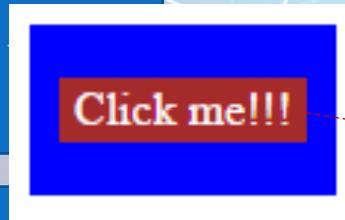
▪ Ví dụ: Cho HTML sau

HTML

```
<div id="divId">  
  <p id="pId">Click me!!!  
</div>
```

CSS

```
div {  
  padding: 2px 10px;  
  color: white;  
  background-color: blue;  
  display: inline-block;  
  cursor: pointer;  
}  
p {  
  background-color: brown;  
  padding: 2px 5px;  
}
```



Click trên vùng này thì sự kiện click trên p hay div sẽ được thi trước?



Lắng nghe sự kiện



- Có 2 cách lan truyền (propagation) sự kiện trong HTML DOM:
 - **bubbling**: sự kiện thành phần **bên trong nhất** được **xử lý trước**, rồi đến các thành phần ngoài.
 - Sự kiện click trên thẻ `<p>` được xử lý trước.
 - **capturing**: sự kiện thành **phần ngoài nhất** được **xử lý trước**, rồi đến các thành phần bên trong.
 - Sự kiện click trên thẻ `<div>` được xử lý trước.



Lắng nghe sự kiện



- Phương thức **addEventListener()** có thể thêm trình lắng nghe sự kiện trên bất kỳ đối tượng HTML DOM như HTML document, Window object, ...
- Ví dụ:

```
var r = document.getElementById("result");  
window.addEventListener('resize', function() {  
    var w = window.innerWidth;  
    var h = window.innerHeight;  
    r.innerHTML = "w: " + w + ", h: " + h;  
});
```



Lắng nghe sự kiện



- Các sự kiện được thêm dùng `addEventListener` thì có thể xóa bằng `removeEventListener`.

```
element.removeEventListener(event, function)
```





Thêm/xóa thành phần HTML

- `document.createElement(e)`
- `document.createTextNode(text)`
- `document.removeChild(e)`
- `document.appendChild(e)`
- `document.replaceChild(e)`

```
var para = document.createElement("p");  
var n = document.createTextNode("Hello");  
para.appendChild(n);
```

```
var d = document.getElementById("divId");  
d.appendChild(para);
```



Hiệu ứng (Animation)

- Các hàm thường được dùng tạo hiệu ứng
 - **setTimeout(<function>, <duration>)**: <function> sẽ được gọi sau khoảng thời gian <duration> (đơn vị mili giây).
 - **setInterval(<function>, <duration>)**: <function> sẽ được gọi sau mỗi khoảng thời gian <duration> (đơn vị mili giây)
 - **clearTimeout(<timeout-variable>)**: xóa một timeout đã thiết lập
 - **clearInterval(<interval-variable>)**: xóa interval đã thiết lập.



Hiệu ứng (Animation)

- Ví dụ: Tạo hiệu ứng một đối tượng img di chuyển từ trái sang phải màn hình.

HTML

```
<div id="container">  
    
</div>  
<button id="btnPlay"  
  onclick="play(this);">Play</button>
```

CSS

```
div {  
  border: 1px solid blue;  
}  
div > img {  
  position: relative;  
}
```



Hiệu ứng (Animation)

```
var timer = undefined, inc = 0;
function play(obj) {
    if (obj.innerText == "Play") {
        obj.innerText = "Stop";
        var container = document.getElementById("container");
        var i = document.getElementById("img");

        timer = setInterval(function() {
            inc += 10;
            if (inc + img.width >= container.clientWidth)
                inc = 0;

            i.style.left = inc + "px";
        }, 120);
    } else if (timer != undefined) {
        obj.innerText = "Play";
        clearInterval(timer);
    }
}
```


JAVASCRIPT

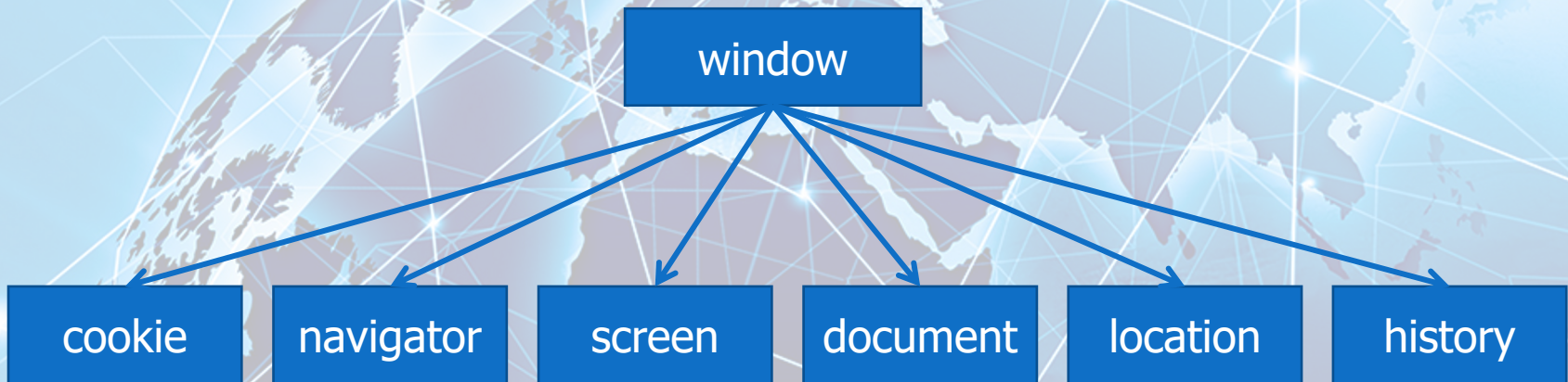
The background of the slide features a stylized world map in a light blue hue. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The overall aesthetic is modern and technological.

HTML BOM



HTML BOM

- BOM (Browser Object Model) cho phép javascript tương tác với trình duyệt.



Có thể sử dụng trực tiếp đối tượng, không cần qua đối tượng window, chẳng hạn dùng window.location hay location là như nhau.



Đối tượng window



- Đối tượng window **đại diện cửa sổ trình duyệt.**
- Tất cả các đối tượng, hàm, biến **toàn cục** tự động sẽ trở thành thành viên của đối tượng window, bao gồm đối tượng document cũng là một thuộc tính của đối tượng window.
- Hai câu lệnh sau là tương đương

```
window.document.getElementById("main")  
document.getElementById("main")
```




Đối tượng window



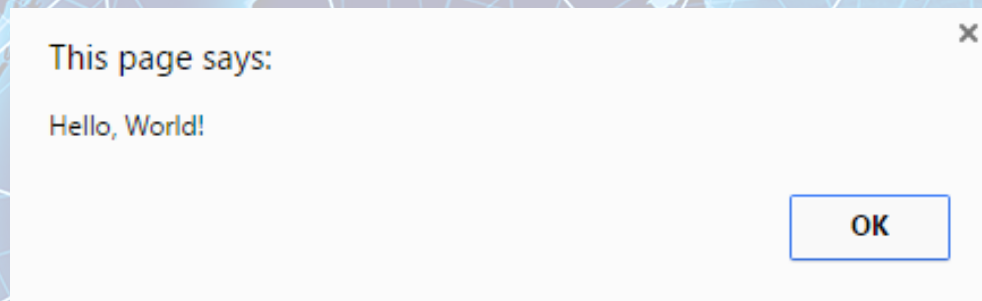
- Lấy kích thước cửa sổ trình duyệt
 - **window.innerHeight**: lấy chiều cao.
 - **window.innerWidth**: lấy chiều rộng.
 - Chú ý: không tính toolbars và scrollbars.
- Các phương thức
 - **window.open()**: mở cửa sổ mới.
 - **window.close()**: đóng cửa sổ hiện tại.
 - **window.moveTo()**: di chuyển cửa sổ hiện tại.
 - **window.resizeTo()**: thay đổi kích thước cửa sổ hiện tại.



Alert Dialog

- Dialog này được sử dụng để hiện một thông báo gì đó đến người dùng.
- Để hiển thị dialog này sử dụng phương thức **window.alert()**.

```
alert("Hello, World!");
```



- Alert box có một button “OK” để thoát.



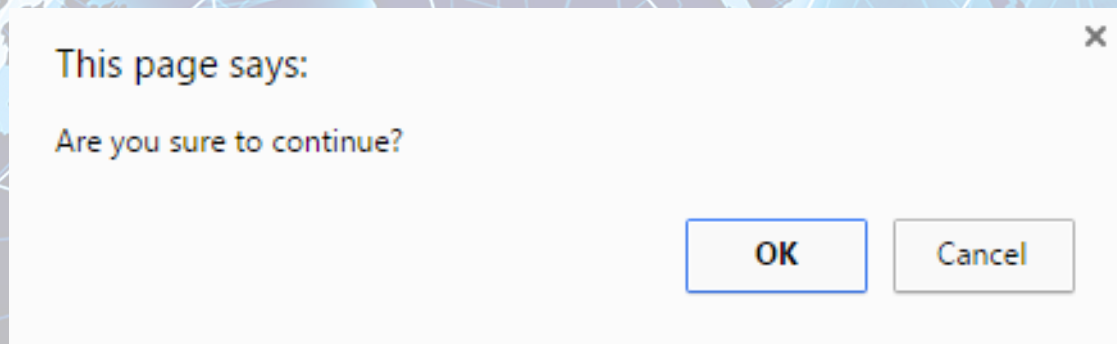
Confirmation Dialog

- Dialog này được sử dụng nếu **muốn nhận một phản hồi** xác nhận một hành động gì đó từ người dùng.
- Để hiển thị dialog này sử dụng phương thức **window.confirm()**.
- Confirmation Dialog có hai button OK và Cancel tương ứng với đồng ý và từ chối một hành động.



Confirmation Dialog

```
var c = confirm("Are you sure to continue?");  
if (c == true) {  
    document.write("User accepted to continue.");  
} else {  
    document.write("User rejected to continue");  
}
```



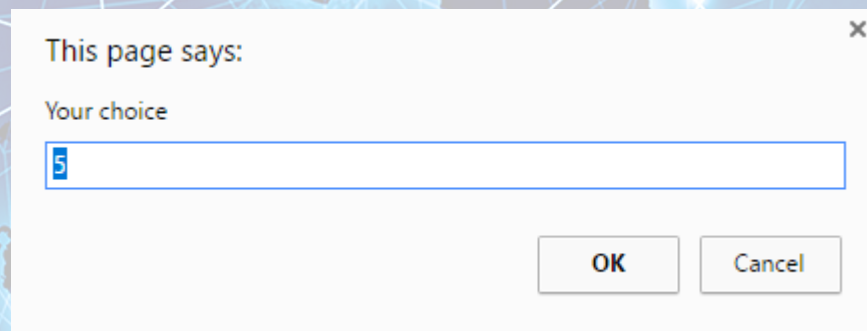


Prompt Dialog



- Dialog này được sử dụng khi cần **tương tác với người dùng**.
- Để hiển thị dialog này sử dụng phương thức **window.prompt()** với hai tham số: nhãn hiển thị và chuỗi giá trị mặc định hiển thị trên textbox.

```
var number = prompt("Your choice", 5);  
document.write(number);
```





Prompt Dialog



- Dialog này có hai button là OK và Cancel
 - Nếu người dùng click OK thì giá trị được nhập vào trong textbox sẽ được trả về.
 - Nếu người dùng click Cancel thì giá trị null được trả về.



Đối tượng window.location

- Đối tượng **window.location** dùng **lấy địa chỉ URL** của trang web hiện tại và chuyển trình duyệt đến trang web mới.
- Sử dụng phương thức **assign()** để chuyển trình duyệt sang trang mới.

```
location.assign("http://localhost:8000/")
```




Đối tượng window.location

- **location.href**: trả về url.
- **location.hostname**: trả về tên miền.
- **location.pathname**: trả về tên đường dẫn.
- **location.protocol**: trả về giao thức.
- **location.port**: trả về cổng.

http://localhost:8000/users/me/

href

protocol

hostname

port

pathname



Đối tượng window.history



- Đối tượng này lưu lại lịch sử duyệt web của người dùng.
- Các phương thức
 - **history.back()**: nạp lại URL trước đó trong danh sách lịch sử duyệt web, tương đương click nút back trên trình duyệt.
 - **history.forward()**: nạp URL tiếp theo trong danh sách lịch sử duyệt web, tương đương click nút forward trên trình duyệt.



Đối tượng window.navigator

- Đối tượng **navigator** chứa **thông tin trình duyệt của người dùng** (visitor).

Thuộc tính/ phương thức	Mô tả
appName	Tên trình duyệt
appCodeName	Tên mã trình duyệt
appVersion	Phiên bản trình duyệt
platform	Nền tảng trình duyệt (hệ điều hành)
language	Ngôn ngữ trình duyệt
userAgent	Tác nhân gửi từ trình duyệt tới server
cookieEnabled	Cookie có đang bật hay không?
onLine	Đang online hay không?
javaEnabled()	Java được bật hay không?



Cookie



- Cookie là dữ liệu được lưu trong tập tin văn bản nhỏ **trên máy tính của client.**
- Khi server gửi dữ liệu đến trình duyệt dạng cookie, nếu trình duyệt chấp nhận thì sẽ được lưu cookie trên ổ cứng.
- Khi trình duyệt gửi yêu cầu trang web đến server thì cookie đó được thêm vào request, điều này giúp server biết thông tin gì đã lưu trước đó.



Cookie

- Các cookie thường gồm các trường:
 - **Expires**: thời điểm cookie hết hạn, mặc định là khi người dùng thoát khỏi trình duyệt.
 - **Domain**: tên miền của trang web.
 - **Path**: đường dẫn tới thư mục hoặc trang web thiết lập cookie.
 - **Secure**: nếu trường này chứa từ "secure" thì cookie chỉ có thể được đọc bởi những server an toàn.
 - **Name=Value**: các cặp key/value của cookie.



Cookie

- Javascript thao tác với cookie thông qua thuộc tính **cookie** của đối tượng document.
- Javascript có thể đọc, tạo, chỉnh sửa, xóa cookie cho trang hiện tại.

// Thêm cookie

```
document.cookie = "username=dhthanh; firstname=thanh;  
lastname=duong; expires=Sun, 28 May 2017 12:00:00 UTC";
```

**// Để xóa cookie chỉ cần thiết lập expires là một thời
điểm quá khứ**

```
var now = new Date();  
now.setMonth(now.getMonth() - 1);
```

```
document.cookie = "expires=" + now.toUTCString() + ";;";
```




Xử lý lỗi

- Có 3 loại lỗi trong lập trình
 - **Lỗi cú pháp (syntax error)**: xảy ra lúc thông dịch trong javascript.
 - **Lỗi thực thi (runtime error)**: còn gọi là ngoại lệ (exception), xảy ra lúc thực thi chương trình (sau thông dịch trong javascript).
 - **Lỗi logic (logical error)**: thường là kết quả lỗi cú pháp và lỗi thực thi.



Xử lý lỗi



- Phiên bản mới nhất của javascript cho phép xử lý lỗi thông qua cơ chế **try...catch...finally**.
- Sau khối lệnh **try** hoặc là khối lệnh **catch** hoặc là khối lệnh **finally** hoặc cả hai.
- Khi có lỗi (thường là lỗi thực thi) xảy ra trong khối **try** thì khối lệnh trong **catch** tương ứng sẽ được thực thi.
- Khối lệnh trong **finally** (nếu có) luôn luôn được thực thi dù có lỗi xảy ra trong khối **try** hay không.



- Cú pháp sử dụng try..catch..finally

```
try {  
    <các-lệnh-try>  
} catch (err) {  
    <các-lệnh-catch>  
} finally {  
    <các-lệnh-finally>  
}
```



Lệnh throw

- Lệnh **throw** được sử dụng để ném ra ngoại lệ, ngoại lệ này có thể là chuỗi, số, boolean, hoặc object.
- Ví dụ:
 throw 500;
 throw "too long";



Ví dụ

- Cho phép người dùng nhập một số, đếm số chữ số của số đó, kết quả in ra console.
- HTML như sau:

```
<input type="text" placeholder="Enter n" id="number" />  
<button onclick="cal();">Calculate</button>
```



Ví dụ

```
function cal() {  
    var n = document.getElementById("number").value;  
    try {  
        n = Number(n);  
        if (isNaN(n))  
            throw "Not number";  
  
        var dem = 0;  
        while (n != 0) {  
            dem++;  
            n = Math.floor(n/10);  
        }  
        console.info(dem);  
    } catch (err) {  
        console.error("Error: " + err);  
    }  
}
```




Đối tượng Error



- Đối tượng **error** cung cấp thông tin lỗi khi có lỗi xảy ra.
- Một đối tượng error có hai thuộc tính
 - **name**: tên lỗi
 - **message**: thông điệp lỗi



Đối tượng Error



- Thuộc tính name của error nhận các giá trị sau:
 - EvalError
 - RangeError
 - ReferenceError
 - SyntaxError
 - TypeError
 - URIError

Q&A